

IronMind Final Project Documentation

Project Information

Project name:

```
```text
IronMind
```
```

Project type:

```
```text
Cloud-hosted fitness tracking backend API
```
```

GitHub repository:

```
```text
https://github.com/jovichen-jc/IronMind.git
```
```

Deployment branch:

```
```text
aws-ecs-deployment
```
```

Live API base URL:

```
```text
http://15.223.186.37:8080
```
```

Executive Summary

IronMind is an ASP.NET Core Minimal API that allows users to register, log in, manage profile information, track meals, track workouts, log hydration, and configure hydration reminders.

The project is deployed on AWS using Docker, Amazon ECR, ECS Fargate, Amazon RDS PostgreSQL, Secrets Manager, and CloudWatch Logs.

The deployed API has been verified through two public endpoints:

- `/health`` confirms the API is running.
- `/db-test`` confirms the API can connect to the AWS RDS database.

Required Deliverables

| Deliverable | Status |
|------------------------|--|
| GitHub Repository Link | Complete |
| Trello Board Link | External link required |
| PDF Documentation | Complete when exported from this document |
| User Guide | Complete |
| Architecture Guide | Complete |
| Database Schema | Complete |
| Docker Files | Complete |
| CI/CD Workflow Files | Complete |
| AWS Deployment Proof | Complete |
| Final Presentation | Separate slide deck or script still required |

Technology Stack

Backend:

```
```text
ASP.NET Core Minimal API, C#/.NET 10
```
```

Database:

```
```text
PostgreSQL with Entity Framework Core
```
```

Authentication:

```
```text
JWT bearer authentication, BCrypt password hashing
```
```

Containerization:

```
```text
Docker
```
```

Cloud:

```
```text
AWS ECR, ECS Fargate, ALB, RDS PostgreSQL, Secrets Manager, CloudWatch Logs
```
```

CI/CD:

```
```text
GitHub Actions
```
```

User Guide

Public API

Base URL:

```
```text
http://15.223.186.37:8080
```
```

This is a direct ECS Fargate task IP. It can change if ECS replaces the task. A load balancer is recommended for a permanent public DNS name.

Health check:

```
```text
GET /health
```
```

Database check:

```
```text
GET /db-test
```
```

```

### ### Local Setup

Run the local database and API:

```
```bash
docker compose up --build
```

Local API:

```
```text
http://localhost:5235
```

Swagger:

```
```text
http://localhost:5235/swagger
```

Main API Features

Authentication:

```
```text
POST /auth/register
POST /auth/login
GET /auth/profile
PUT /auth/profile
```

Nutrition:

```
```text
POST /nutrition/meals
GET /nutrition/meals/summary
PUT /nutrition/meals/{id}
DELETE /nutrition/meals/{id}
GET /nutrition/food/search?q=banana
```

Exercise:

```
```text
POST /exercise/cardio
POST /exercise/strength
GET /exercise/history
PUT /exercise/cardio/{id}
PUT /exercise/strength/{id}
DELETE /exercise/{id}
```

Hydration:

```
```text
POST /hydration/water
GET /hydration/water/summary
POST /hydration/reminders
PATCH /hydration/reminders/{id}?active=false
```

Protected endpoints require:

```
```text
Authorization: Bearer <jwt-token>
```

## ## Architecture Guide

### ### Runtime Architecture

```
```text
User / API Client
  -> ECS Fargate container on HTTP port 8080
  -> RDS PostgreSQL database
```

AWS Services

Service	Purpose
ECR	Stores the Docker image
ECS Fargate	Runs the API container
Application Load Balancer	Optional improvement for stable public HTTP URL
RDS PostgreSQL	Stores application data
Secrets Manager	Stores database and JWT secrets
CloudWatch Logs	Stores container logs
IAM	Controls AWS permissions

Application Layers

```
```text
IronMind.API Routes, auth config, DI, Swagger, health checks
IronMind.Core Models, DTOs, interfaces, shared helpers
IronMind.Data EF Core DbContext, migrations, repositories
IronMind.Services Business logic
IronMind.Tests Unit tests
```

## ## Database Schema

### ### Entity Relationships

```
```text
Users
├── MealLogs
├── WorkoutLogs
│   ├── CardioDetails
│   └── StrengthSets
├── WaterLogs
└── ReminderSchedules
```

Main Tables

Users:

- `Id`
- `Email`
- `PasswordHash`
- `Name`
- `DateOfBirth`

- `WeightKg`
- `HeightCm`
- `Units`
- `DailyCalorieGoal`
- `DailyWaterGoalML`
- `DeviceToken`
- `CreatedAt`

MealLogs:

- `Id`
- `UserId`
- `FoodName`
- `Calories`
- `ProteinG`
- `CarbsG`
- `FatG`
- `OpenFoodFactsId`
- `LoggedAt`

WorkoutLogs:

- `Id`
- `UserId`
- `Type`
- `LoggedAt`
- `Notes`

CardioDetails:

- `Id`
- `WorkoutLogId`
- `ActivityType`
- `DurationMinutes`
- `DistanceKm`
- `CaloriesBurned`

StrengthSets:

- `Id`
- `WorkoutLogId`
- `ExerciseName`
- `SetNumber`
- `Reps`
- `WeightKg`

WaterLogs:

- `Id`
- `UserId`
- `AmountML`
- `LoggedAt`

ReminderSchedules:

- `Id`
- `UserId`
- `IntervalMinutes`
- `StartTime`
- `EndTime`
- `IsActive`

Docker Files

Dockerfile:

```
```text
IronMind.API/Dockerfile
```
```

Docker Compose:

```
```text
docker-compose.yml
```
```

Docker ignore:

```
```text
.dockerignore
```
```

CI/CD Workflow Files

Deployment workflow:

```
```text
.github/workflows/deploy.yml
```
```

Pull request validation workflow:

```
```text
.github/workflows/pr-validation.yml
```
```

AWS Deployment Proof

Live Health Check

URL:

```
```text
http://15.223.186.37:8080/health
```
```

Verified response:

```
```json
{
 "status": "healthy",
 "service": "IronMind API",
 "time": "2026-05-15T01:21:39.5784579Z"
}
```
```

Live Database Check

URL:

```
```text
http://15.223.186.37:8080/db-test
```
```

Verified response:

```
```json
{
 "database": "Connected",
 "serverTime": "2026-05-15T01:21:40.6798661Z",
 "message": "Database connection successful"
}
```
```

AWS Resource Names

ECR repository:

```
```text
481088927864.dkr.ecr.ca-central-1.amazonaws.com/ironmind/api
```
```

ECS cluster:

```
```text
ironmind-cluster
```
```

ECS service:

```
```text
ironmind-api-service
```
```

Public endpoint:

```
```text
http://15.223.186.37:8080
```
```

RDS instance:

```
```text
ironmind-postgres
```
```

Secrets:

```
```text
ironmind/prod/connection-string
ironmind/prod/jwt-secret
ironmind/prod/jwt-issuer
ironmind/prod/jwt-audience
```
```

Cost Note

The AWS resources are billable while they exist. The main billable resources are :

- RDS PostgreSQL
- ECS Fargate
- CloudWatch Logs
- Secrets Manager

For cleanup after grading, stop or delete unused resources.

Conclusion

IronMind satisfies the core cloud deployment requirements:

- The API is containerized with Docker.
- The image is stored in ECR.
- The API runs on ECS Fargate.
- Traffic reaches the API through a public ECS Fargate task endpoint.
- Production secrets are stored in Secrets Manager.
- The API connects successfully to RDS PostgreSQL.
- CI/CD workflow files are included in the GitHub repository.